OpenTwin Interview Stack – Technical Specification

Oct 30, 2026 · @Someone · BaselHack · Draft v0.1

1. Purpose, scope and goals

OpenTwin beats Tavus not on photorealism but on the job: a customer twin built from the sprint context in 15 minutes, interviewed live while it reads the prototype, with findings landing on the sprint board automatically. Tavus is a general real-time video API; OpenTwin is a research instrument assembled from open-source components.

In scope: twin creation from sprint artefacts, real-time voice and avatar conversation, screen and prototype understanding, meeting integration (own web room first, Teams/Zoom second), interviewer copilot, insight synthesis and export to Miro.

Out of scope for v1: photoreal clones of real customers without written consent, quantitative market sizing, replacing real-customer validation.

Goal	Measure	Target v1
Twin from context, fast	Time from sprint artefacts to interview-ready twin	⩽ 15 min, ⩽ 30 min with voice clone
Natural conversation	Voice-to-voice response latency	p50 ⩽ 1.0 s, p95 ⩽ 1.5 s
Sees what the interviewer shows	Correct answer to "what do you see / where would you click"	⩾ 90 % on a 20-item probe set
Grounded, not invented	Twin statements traceable to a source or flagged as assumption	⩾ 80 % traceable
Not a yes-machine	Weak concepts the twin rejects or questions	⩾ 1 in 2 of seeded weak concepts
Closes the loop	Insights on the Miro board after the session ends	⩽ 5 min, no manual copy
Sovereign option	Share of components self-hostable in CH/EU	100 % in stack B

Targets are engineering goals for the hackathon and pilot, not measured results.

2. Benchmark: Tavus CVI and the gaps we exploit

Tavus wins on the face and on integration speed; it has no twin factory, sees the screen only as pixels, and stops at the transcript. Its Conversational Video Interface bundles perception, STT, LLM, TTS and face rendering in one pipeline with a bring-your-own-LLM

option. Raven-1 reads a shared screen without extra configuration and can run analysis queries at the end of a call.

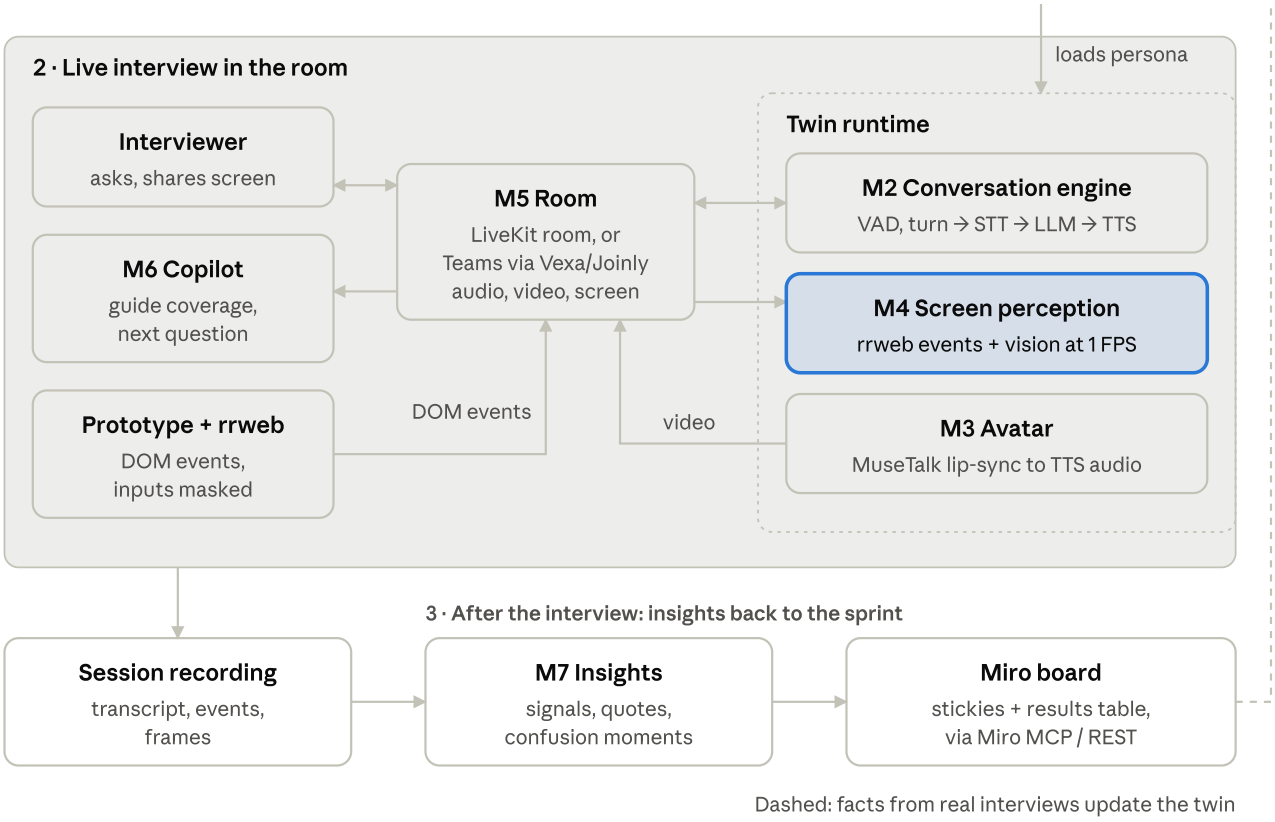
Capability	Tavus CVI today	OpenTwin answer
Persona creation	Persona Builder: system prompt, knowledge base, memory, written by hand	**Twin Factory** compiles the persona from sprint artefacts with source links and a readiness score
Realism of the face	Phoenix-4, photoreal replica from a short video	Open source lip-sync (MuseTalk) on a reference loop; stack C plugs in a commercial avatar layer
Turn-taking	Sparrow-1 timing model	Silero VAD + open smart-turn / turn-detector models
Screen understanding	Raven: rolling frames of the shared surface (pixels)	**Dual channel**: DOM events from the prototype (rrweb) plus vision frames as fallback; optional agent mode where the twin clicks itself
Research workflow	None: generic conversation API	Interview guide, interviewer copilot, synthesis per sprint question
Results	Transcript and perception summary via API	Structured insights pushed to Miro (and repository) automatically
Hosting and data	US SaaS, usage-based pricing	Self-hostable in CH/EU (stack B), no per-minute fees
Meetings	Daily rooms, React components	Own room (LiveKit/Daily) plus Teams/Zoom/Meet bots via Vexa or Joinly

Where we do not try to win in v1: photoreal faces and sub-second latency at scale. We target parity on latency and accept a visibly synthetic but expressive avatar.

3. Solution overview and reference architecture

OpenTwin runs in three phases: the Twin Factory builds the persona from the sprint before the interview, the twin runtime joins a live room and reads the prototype, and the insight module writes results back to the board.





Read it top to bottom: everything left of the twin runtime is the human side of the room; the twin itself is three swappable services (conversation, screen, avatar) fed by one persona card.

4. Stack combinations

Recommendation: build stack A at the hackathon, keep every interface stack-B compatible, and use stack C for customer demos. All three share the same open-source core (orchestration, Twin Factory, screen events, insights); only the model layers swap.

- **A – OSS core + hosted models (hackathon):** fastest path, one GPU for the avatar, hosted speech-to-speech model.
- **B – Sovereign, fully self-hosted:** every component open source and runnable in a Swiss or EU data centre; no per-minute fees; more GPU and tuning effort.
- **C – Hybrid best experience:** OSS core and our moats (Twin Factory, screen events, insights) with best-in-class commercial voice and face where the user feels the difference.

Layer	A – Hackathon	B – Sovereign	C – Best experience
Room / transport	LiveKit Cloud or Daily	LiveKit server, self-hosted	LiveKit Cloud
Orchestration	Pipecat	Pipecat or LiveKit Agents	Pipecat
Turn-taking	Silero VAD + smart-	Silero VAD + smart-turn	Model-native barge-in

	turn	/ LiveKit turn detector	+ smart-turn
Speech in	Gemini Live (native audio)	faster-whisper or Mistral Voxtral	OpenAI Realtime or Gemini Live
Persona brain (LLM)	Gemini Live, persona prompt + tools	Qwen3 or Mistral via vLLM	gpt-realtime or Gemini; Claude/GPT for synthesis
Voice out / clone	Gemini voices; Kokoro for stock voice	Chatterbox, XTTS-v2 or F5-TTS (clone from consented samples)	ElevenLabs clone or Cartesia
Avatar	MuseTalk on 1 GPU	MuseTalk or Duix, self-hosted	Anam, Simli or HeyGen LiveAvatar (LITE)
Screen: events	rrweb snippet in prototype	rrweb	rrweb
Screen: vision	Gemini Live frames (1 FPS)	Qwen3-VL + OmniParser	Gemini Live or OpenAI image input
Screen: agent mode	Playwright MCP	Playwright MCP / Browser Use	Playwright MCP
Twin Factory	LLM pipeline (genagents pattern) + LlamaIndex	Same, local LLM + embeddings	Same, frontier LLM
Twin memory	JSON persona + vector store	Letta or Mem0 self-hosted	Letta or Zep
Meeting bridge	Own web room only	Vexa self-hosted	Joinly or Recall.ai
Insights and export	LLM synthesis + Miro REST/MCP	Same + n8n self-hosted	Same + Dovetail
Observability	Langfuse	Langfuse self-hosted	Langfuse

Where a component name links, it was checked for this spec; the rest are well-known projects listed in the market analysis.

5. Module specifications

Seven modules, each a separate service behind the event bus, so any model can be swapped without touching the others.

M1 Twin Factory (context → twin)

Turns sprint artefacts into an interview-ready persona in ⩽ 15 minutes; this is the main moat.

1. **Ingest:** Miro board (via Miro REST/MCP), expert-interview transcripts (VTT), sprint

challenge and target-customer note, prototype URL, optional real-customer transcripts.

2. **Extract:** LLM extracts facts per persona slot (demographics, context of use, jobs, pains, current tools, attitudes, vocabulary) with a source id per fact.
3. **Life story:** generate a first-person "interview transcript" of the persona's life and habits, following the interview-based twin pattern of Stanford genagents; every invented detail is marked `assumption`.
4. **Compile:** persona card (JSON, see section 6), system prompt, behaviour rules, knowledge index (LlamaIndex).
5. **Voice and face:** stock voice + stock avatar by default; a clone only from consented samples (section 8).
6. **Readiness score:** 20 probe questions answered 3 times each; score = consistency × grounding. Below 0.7 the factory asks the sprint team 3 targeted questions to fill gaps.

M2 Conversation engine

A Pipecat pipeline that makes the twin speak like a customer, not an assistant.

- Full duplex with barge-in; end-of-turn by Silero VAD + smart-turn model.
- Behaviour rules: answers from own life and experience, admits not understanding, hesitates, does not volunteer design advice, may disagree; language de-CH, de, en.
- Tools the LLM can call: `get_screen_state()`, `recall(topic)`, `note_reaction(kind, intensity)`, `look_closer(region)` (triggers a high-res vision frame).
- Persona drift guard: every 5 turns a cheap check against the persona card; on drift the next turn gets a corrective system note.

M3 Avatar renderer

Gives the twin a face in the room at 25 fps.

- MuseTalk lip-syncs a 10–20 s reference video loop (idle, listening, nodding states) to the TTS audio; output is a WebRTC video track.
- Listening state while the interviewer speaks; head nod on backchannel.
- Fallback when no GPU: audio-only with a static portrait and a speaking indicator.
- Stack C: replace with an avatar-layer API (Anam, Simli, LiveAvatar LITE) behind the same interface.

M4 Screen perception (dual channel)

The twin knows what is on screen from structured events first and pixels second.

- **Event channel:** an rrweb snippet in the prototype (Lovable, v0, any web build) streams DOM mutations, clicks, scrolls and inputs (inputs masked). A summariser keeps a `ScreenState`: page title, visible headings, buttons, last action, errors, at 2 Hz.

- **Vision channel:** the interviewer's screen-share track sampled at 1 FPS, only on change (perceptual hash); captioned by Gemini Live or Qwen3-VL; used for Figma prototypes, images and anything without DOM.
- **Agent mode (optional):** the twin "takes the mouse" in a sandboxed browser via Playwright MCP and thinks aloud; useful for first-click and navigation tasks.
- Merge rule: events win on facts (what is there), vision wins on look (layout, colour, imagery).

M5 Session and meeting layer

One interview = one room with interviewer(s), the twin and optional observers.

- Default: own LiveKit room in the browser, link shared in the sprint; the twin publishes audio + avatar video and subscribes to screen share.
- Teams/Zoom/Meet: a Vexa or Joinly bot joins as participant and relays audio, avatar video and screen share to the engine (see risk R1).
- Recording of all tracks plus the event stream, with consent banner.

M6 Interviewer copilot

The wish from the kick-off: a coach for team members who have never run an interview.

- Side panel with the interview guide, coverage per question and the next suggested follow-up ("ask why it is a 4 and not a 5").
- Flags leading questions and pitching; visible only to the interviewer.

M7 Insights and feedback loop

Results land where the sprint works, minutes after the call.

- Inputs: timestamped transcript, `ScreenState` timeline, reactions, perception notes.
- Output per sprint question: signal (supports / contradicts / unclear), top quotes, confusion moments with screenshot, follow-up questions for real customers.
- Export: Miro sticky notes per persona and question plus a results table via Miro REST/MCP; optional n8n flows to Notion, Teams or Dovetail.
- Twin update: after a real-customer interview, new facts go into the persona with source `real_interview`, raising grounding over time.

6. Data model, interfaces and APIs

Three objects carry everything: the Persona, the Session event stream and the Insight report. Services talk over an event bus (Redis Streams for A, NATS for B) and a small REST/WebSocket API.

Persona card (excerpt)

```
{
  "id": "twin_7f3a",
  "sprint_id": "sprint_2026_11_ebike",
  "version": 3,
  "identity": {"name": "Reto", "age": 46, "location": "Baar ZG", "language":
"de-CH"},
  "context": {"job": "...", "household": "...", "tools_today": ["..."]},
  "jobs": [{"text": "...", "source": "miro:sticky_123"}],
  "pains": [{"text": "...", "source": "assumption"}],
  "attitudes": {"tech_affinity": 2, "price_sensitivity": 4,
"trust_in_new_brands": 2},
  "speech_style": {"register": "informal", "verbosity": "short", "fillers":
true},
  "life_story_ref": "kb://twin_7f3a/story",
  "voice": {"provider": "kokoro", "voice_id": "stock_m_02", "cloned": false},
  "avatar": {"renderer": "musetalk", "reference": "stock_m_02.mp4", "cloned":
false},
  "readiness": {"score": 0.78, "consistency": 0.86, "grounding": 0.91}
}
```

Session events

Event	Producer	Payload (key fields)
`utterance.final`	STT / S2S model	speaker, text, t_start, t_end
`screen.state`	M4 event channel	url, title, visible_elements[], last_action, errors[]
`screen.caption`	M4 vision channel	frame_id, caption, regions[]
`twin.reaction`	M2 tool call	kind (confused, pleased, sceptical), intensity 1–5, t
`copilot.suggestion`	M6	question_id, suggestion
`session.ended`	M5	duration, recording_refs[]

API

Method	Endpoint	Purpose
POST	`/twins`	Create twin from sources (Miro board id, files, URLs); returns job id
GET	`/twins/{id}`	Persona card, readiness score, open gap questions
POST	`/twins/{id}/answers`	Sprint team answers gap questions; recompiles
POST	`/sessions`	Open room for a twin + interview guide; returns join links

WS	`/sessions/{id}/events`	Live event stream (copilot, dashboards)
POST	`/sessions/{id}/end`	Close, trigger synthesis
GET	`/sessions/{id}/insights`	Insight report (JSON + Markdown)
POST	`/insights/{id}/export`	Push to Miro board / n8n webhook

An MCP server wraps the same endpoints, so an assistant in Miro, Teams or Claude can create twins and fetch insights.

7. Latency budget, infrastructure and cost

The 1.0 s voice-to-voice target is reachable only with streaming at every stage and the avatar rendering on the same GPU host as TTS. Budgets below are design allocations to measure against, not benchmarks.

Stage	Component (stack B)	Budget p50	Technique
End of turn detected	Silero VAD + smart-turn	250 ms	Semantic end-of-turn instead of fixed silence timeout
Final transcript	faster-whisper streaming	150 ms	Partial transcripts feed the LLM early
First LLM token	vLLM, persona prompt cached	300 ms	Prefix caching of persona and screen state
First audio	Kokoro / XTTS streaming	150 ms	Sentence-level streaming, first clause only
First avatar frame	MuseTalk	80 ms	Pre-loaded reference latents, audio chunked at 40 ms
Network (WebRTC)	LiveKit	70 ms	Server in same region as users (CH/EU)
Total		**1,000 ms**	

In stack A and C, a speech-to-speech model (Gemini Live, OpenAI Realtime) replaces the transcript, LLM and audio stages with one streamed call. OpenAI's Realtime API is priced at about $32 per million audio input tokens and $64 per million output tokens; open models in stack B remove per-minute fees but add GPU cost.

Sizing (estimates to validate in the first load test):

- Hackathon (A): 1 GPU with 24 GB (e.g. L4 or A10) for MuseTalk + stock TTS, 1–2 parallel sessions; everything else on CPU or hosted APIs.
- Pilot (B): 1 GPU node per 2–4 parallel sessions for avatar + TTS + STT, plus a shared LLM node (e.g. 1× 80 GB GPU for a 30B-class model via vLLM); Swiss or EU region

LLM node (e.g. 1× 80 GB GPU for a 30B-class model via vLLM); Swiss or EU region.

- Storage: recordings and event streams per session in object storage, 30-day default retention (section 8).

8. Security, privacy and consent

Default twins are fictional composites with stock voice and face; anything modelled on a real person needs written consent, and every session discloses that participants talk to an AI.

- **Legal frame:** Swiss revised Data Protection Act (nDSG/FADP) and GDPR for EU participants; AI-interaction and deepfake transparency duties under the EU AI Act (Art. 50) – check the dates that apply at launch, as the EU has proposed changes to the timeline.
- **Likeness and voice:** clones only of team members or customers who signed a consent form covering purpose, duration and deletion; clone assets stored encrypted, deletable on request; no clone of a third party from public material.
- **Disclosure:** "AI twin" label on the avatar tile and in the room name; synthetic findings marked as synthetic in every export.
- **Prototype confidentiality:** screen frames and DOM events stay inside the session; rrweb runs with `maskAllInputs` and class-based blocking for sensitive areas; no frames sent to model providers in stack B.
- **Data residency:** stack B runs fully in a Swiss or EU region; stacks A and C must list each provider and region in the data processing record.
- **Retention:** recordings and events 30 days by default, insights kept with the sprint; configurable per customer.
- **Access:** SSO (OIDC) for sprint teams, per-sprint workspaces, audit log of twin creation and exports.
- **Lesson from the market:** screen-reading assistants have leaked screenshots and transcripts before (Cluely, 2025); treat screen data as the most sensitive asset.

9. Quality and validation

The product only matters if twins are both immersive and honest, so we test fidelity, sycophancy and team impact, not just latency. Critics document that synthetic users answer too positively (Indeemo); our test plan targets exactly that.

Test	Method	Pass criterion
Consistency	20 probe questions × 3 runs per twin	≥ 0.85 agreement on facts and attitudes
Grounding	Classify each twin statement as sourced / assumption / invented	≥ 80 % sourced or flagged

Screen comprehension	20 scripted screens (web + Figma), ask what the twin sees and where it would click	≥ 90 % correct
Sycophancy	Seed 4 weak concepts among 8	Twin rejects or questions ≥ 2 of 4 weak ones
Calibration vs real customers	Same interview guide with 3–5 real customers and the twin	Twin surfaces ≥ 60 % of the themes found with real customers
Immersion	Sprint team rates live twin vs text persona report (A/B, 7-point scale)	Live twin scores higher on "I understood the customer"
Decision impact	Count sprint decisions changed after a twin session	≥ 1 per sprint in pilot
Latency	Logged p50/p95 per stage (Langfuse + LiveKit stats)	p50 ≤ 1.0 s, p95 ≤ 1.5 s

Regression suite: every persona and prompt change re-runs consistency, grounding and sycophancy tests in CI.

10. Hackathon build plan and roadmap

Four parallel tracks over the weekend, mirroring the buckets from the kick-off, with one integration point: the live demo. Interfaces from section 6 are agreed first, so each track can mock the others.

Track	Builds	Stack A components	Demo checkpoint
1 Twin data	M1 Twin Factory, persona card	LLM pipeline, LlamaIndex, Miro export	Twin card with readiness score from real sprint material
2 Audio and video	M2 conversation, M3 avatar	Pipecat, Gemini Live, MuseTalk, LiveKit	Twin speaks with a face in the room, p95 ≤ 1.5 s
3 Screen	M4 screen perception	rrweb snippet, Gemini frames, Playwright MCP	Twin names what is on screen and reacts to a click
4 Loop and demo	M5 room, M6 copilot (stretch), M7 insights	LiveKit room, LLM synthesis, Miro REST	Stickies on the board within 5 min after the call

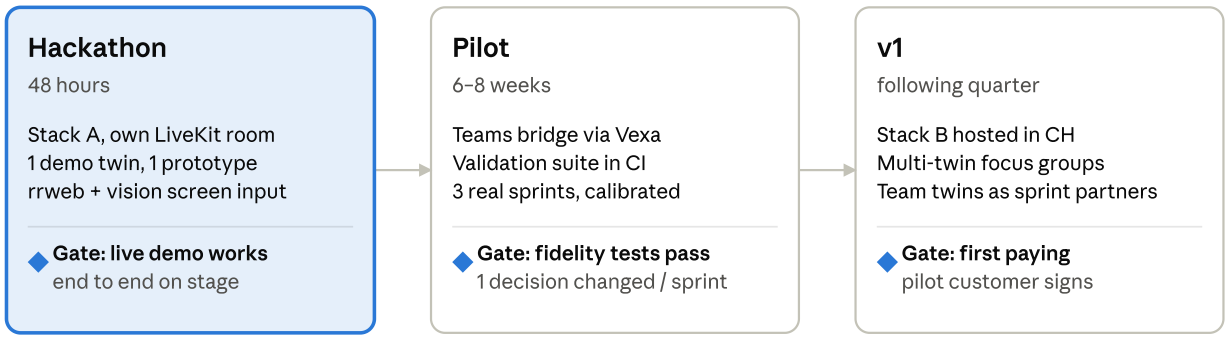
Demo script (5 minutes):

1. Point the Twin Factory at the sprint's Miro board; the twin card appears with its readiness score.

2. Interview the twin live while sharing the prototype; it reacts to what it sees.
3. End the call; insights appear on the Miro board.
4. Show the same question answered by a text-only persona report, to make the immersion gap visible.

From a 48-hour demo to a sovereign product in three gated phases

Durations after the hackathon are a proposal



Each phase ends at a gate; the pilot starts only if the hackathon demo ran end to end.

11. Risks, open questions and decisions

The biggest risks are getting avatar video into Teams, the uncanny look of open-source avatars, and twins that please instead of challenge.

ID	Risk	Impact	Mitigation
R1	Publishing avatar video from a bot into Teams/Zoom is limited or slow	Twin only as audio in corporate calls	Own LiveKit room as default; test Vexa/Joinly video early; Teams media bot as later track
R2	Open-source lip-sync looks uncanny next to Tavus	Lower immersion	Stylised or semi-real reference video; stack C avatar layer for demos
R3	Twin is too positive or generic	False confidence in prototypes	Behaviour rules, sycophancy tests, real-customer calibration
R4	No GPU available at the hackathon	No avatar	Book a cloud GPU in advance; audio + portrait fallback
R5	Swiss German speech recognition errors	Broken conversation	Interview in High German or English for v1; evaluate STT on dialect samples
R6	Figma prototypes expose no DOM	Screen channel falls back to vision only	Figma MCP for structure + vision captions

R7	Consent and likeness misuse	Legal and trust damage	Stock avatars by default, consent flow, audit log
R8	Scope creep in 48 h	Nothing demo-ready	Demo path fixed on day 1 (section 10)

Decisions needed before the hackathon starts:

- [ ] Stack for the hackathon: A as proposed, or B from the start?
- [ ] Demo setting: own web room or Teams?
- [ ] Interview language for the demo: High German or English?
- [ ] Stock avatar or consented clone of a team member?
- [ ] Which sprint case and prototype do we use as the demo scenario?

Open questions: Which vendor did the kick-off mean by "Targo/Targos"? Is there budget for a cloud GPU and hosted model APIs during the weekend?

12. Sources

- Tavus docs: Screen Share with Raven
- Tavus: Raven-1 launch
- Google Cloud: Gemini Live API overview
- Pipecat (GitHub) and LiveKit Agents (GitHub)
- Fora Soft: lip-sync and avatar models in production (MuseTalk real-time)
- Spheron: self-hosted avatar generator (MuseTalk, Duix, MagiHuman)
- Open-source AI avatar system (Whisper, Chatterbox, MuseTalk)
- Synthesia: real-time interactive avatar platforms 2026
- Vexa: open-source meeting bot API
- Joinly: AI agents as meeting participants